Response to Reviewers — YOLOv8-FLEO (MDPI Algorithms)

Working document. Each point: the reviewer's ask, our status, and the paste-ready reply. **[ACTION]** marks work that must be done before the reply is truthful.

Status legend: ready · needs a GPU run · needs a code fix + rerun · figure work

---

Reviewer 1

R1-1 — Justify subspace width d = 8

Hardware-cost half is computed and final (see `docs/reviewer_response_d_ablation.md`): d=4/8/16/32 → 0.50× / 1.00× / 2.05× / 4.30× FLEO footprint. Accuracy half needs the four RAF-DB runs (`notebooks/ablation_d.ipynb`). Reply paragraph is drafted there.

R1-2 / R1-3 (and R3-1) — How are the fuzzy labels built? α and π?

Reviewer: RAF-DB/FER2013 are single-label; where do the annotator votes n_c in Eq. (2) come from — multi-annotator metadata, or algorithmic soft labels? Justify α and the prior π.

Honest status. The benchmarks are single-label; there is **no per-image vote metadata**. In the released training path the classification term currently uses the standard (hard-label) detector loss plus the orthogonality and binding auxiliaries — i.e. **Eq. (2) is described but not yet exercised in the runs**. This must be fixed before the reply below is true.

[ACTION] Make the code match the paper: enable additively-smoothed soft targets during training (e.g. ultralytics `label_smoothing = α`, optionally with a class-frequency / confusion prior π) and re-run. This folds into the retraining already required by R1-1 and R3-4.

Reply (valid once the above is done):

> The benchmarks are single-label and we use **no** multi-annotator metadata. Eq. (2) is the general membership definition, where n_c is the vote count for class c. For a single-label corpus n_c = 𝟙[c = y], so Eq. (2) reduces to additively-smoothed soft targets **μ_c = (𝟙[c=y] + α·π_c) / (1 + α)** — generated **algorithmically and deterministically** from the ground-truth label, the smoothing strength α, and the prior π, with no annotator votes required. The construction is a few lines and is released as part of the training script, so the fuzzy targets are fully reproducible. We set **α = [FILL]** and **π = [FILL, e.g. the empirical class-frequency prior]**, so probability leaks toward frequently-confused neighbours while confident, well-

> represented classes stay near one-hot. We have clarified this in Sec. 3.1.1 and corrected the wording so Eq. (2) is presented as the general form that degenerates to smoothed labels on single-label data.

R1-5 — English + reference formatting (copy-edit pass, do last)

Reviewer 2 (mostly cosmetic — do in one editing pass)

- **Title ≤ 2 lines** — shorten to e.g. "DPU-Native YOLOv8-FLEO: Fold-Out Orthogonalization for Real-Time FPGA Facial-Expression Recognition".
- **Delete** the "remainder of this paper …" paragraph (lines 100-103) .
- **Unify terminology** → pick one, "YOLOv8-FLEO framework", everywhere (not module/model) .
- **Figures 1-4 X/Y symbols** — check and disambiguate .
- **Redraw Fig 1** per the itemised edits; merge Fig 2 into Fig 1; vector graphics; enlarge small text; drop training flows from architecture panels .
- **Redraw Fig 3 & Fig 5** as academic (more imagery, less text, not colour-only) .
- **Trim Table 2/3 "Value" column** (e.g. "SGD (Momentum=0.937)" → "SGD") .
- **New "Evaluation Metrics" section + a heatmap figure** / .
- **Compare with YOLO11 and YOLO26** — needs training runs.
- **Spelling**: `bottleneckssuch` → `bottlenecks such`, `recurrenceand` → `recurrence and`, `workarounds` check .
- **Affiliations** of authors 1 and 3 — split combined affiliations .

Reviewer 3

R3-1 — reproducibility of fuzzy targets

Same as R1-2/R1-3 above — one shared reply.

R3-2 — "GS is a training-time regularizer" contradicts Table 7 (0.858 → 0.073) CRITICAL

Reviewer: Removing GS collapses RAF-DB accuracy 0.858 → 0.073; recovered to 0.849 only after 10 fine-tuning epochs. So GS is load-bearing in the forward path, not a mere regularizer. Clarify.

Honest reframe (the paper's wording must change): GS is not a no-op at inference. The correct claim is a **two-step deployment**: (i) fold-out removes the DPU-hostile GS operator, which does break the forward path (→ 0.073); (ii) a short **post-fold fine-tuning** (10 epochs, no orthogonality) lets the DPU-native convolutions re-absorb the function GS was performing (→ 0.849). So the benefit is internalized into the weights by fine-tuning, not "already free". We will rewrite Sec. 3.1.3 and the abstract to state this explicitly and stop calling GS a pure training-time regularizer.

R3-3 — add suggested citations (add the 4 references)

R3-4 — control: does baseline gain from the same 10 epochs? CRITICAL

Reviewer: the 0.073→0.849 recovery may be extra optimization, not orthogonalization. Give the control: standard YOLOv8 + the same 10 epochs / schedule.

[ACTION] Run: baseline YOLOv8n + 10 extra epochs at η0 = 5e-4 cosine, no orthogonality, same data. Report Δ vs the fine-tuned FLEO. **This is decisive**: if the baseline gains as much, the FLEO contribution is not supported and the framing must be honest about it.

Work queue (what unblocks the most replies)

1. **Add label-smoothing (Eq. 2) to training** → unblocks R1-2/3, R3-1.
2. **Retrain the RAF-DB matrix** (baseline, FLEO, d∈{4,8,16,32}, +10-epoch controls) → unblocks R1-1, R3-4, and provides the numbers R3-2 relies on.
3. **Train YOLO11 + YOLO26** on RAF-DB → R2 comparison.
4. **One editing pass** for all R2 cosmetic + R1-5 + R3-3.
5. **Redraw Figures 1/2/3/5 + heatmap.**

Reviewer 1 — Comment 1: justification of subspace width d = 8

> "The rationale for setting subspace dimension d=8 is not provided. Please add ablation experiments evaluating algorithm accuracy and hardware cost under different d values to justify the selection of d=8."

This note bundles everything needed to answer it: the hardware-cost half (computed analytically from the architecture — no training), the accuracy half (run on GPU), and the ready-to-paste response paragraph.

---

1. Where d lives in the code

`d` = `subspace_dim` = per-emotion subspace width. In the paper's model (`fleo/fleo_block.py`, main branch) it is the constructor argument `d`, **default 8**, and it sets the FLEO projection width `mid = K·d = 7d`. It is exposed on the CLI as `scripts/train.py --d`. (The older `models/fleo_module.py` on other branches defaults to 16/32 and is **not** the paper's model — archive it to avoid confusion.)

`d` has nothing to do with INT8 — that is the 8-bit quantization word length, a separate hardware setting. The shared "8" is a coincidence.

---

2. Hardware cost vs d (computed, exact)

The deployed FLEO block at each neck site is `proj(1×1) + gate_in(1×1) + gate_fc(1×1) + fuse(3×3)`, all of width `mid = 7d` (Gram–Schmidt / Householder / binding head are training-only or route-folded, so they do not run on the DPU). Parameter count per site: `11·c1·mid + mid² + 5·mid + 2·c1`.

Representative YOLOv8n neck sites P3 (c1=128) and P4 (c1=256):

d	channels (7d)	FLEO params (P3+P4)	relative to d=8	added over YOLOv8n (~3.0 M)
4	28	120,888	0.50×	+4%
8	56	**244,144**	**1.00×**	**+8%**
16	112	500,064	2.05×	+17%
32	224	1,049,536	4.30×	+35%

MAC/FLOP cost follows the same ratios (conv cost ∝ mid). **Cost roughly doubles each time d doubles.**

---

3. Accuracy vs d (fill after the GPU runs)

d	mAP50	mAP50-95	macro-F1
4	_	_	_
8	_	_	_
16	_	_	_
32	_	_	_

Produced by `notebooks/ablation_d.ipynb` (see §5). Expected shape: accuracy rises from d=4→d=8, then saturates — which is exactly what justifies d=8.

---

4. Response paragraph (paste into the rebuttal, fill the blanks)

> We thank the reviewer. We have added an ablation over the per-emotion subspace width **d ∈ {4, 8, 16, 32}** on RAF-DB (new Table X), reporting both recognition accuracy and hardware cost.
>
> **Hardware cost.** FLEO projects each of the two neck sites (P3, P4) to K·d = 7d channels, so its parameter and MAC cost grow approximately linearly in d, doubling with each doubling of d. Relative to d = 8, the FLEO footprint is 0.50× at d = 4, 2.05× at d = 16, and 4.30× at d = 32 — adding 4%, 8%, 17% and 35% respectively over the 3.0 M-parameter YOLOv8n backbone.
>
> **Accuracy.** [d=4: **, d=8:** , d=16: __, d=32: __ mAP50]. Recognition accuracy improves from d = 4 to d = 8 but saturates beyond d = 8 (Δ ≤ __ points for d = 16 and d = 32). Consequently, **d = 8 lies at the knee of the accuracy–cost trade-off**: it captures the representational benefit of the orthogonal subspaces while keeping the accelerator footprint minimal. We therefore adopt d = 8 for all reported experiments, and report the full sweep in Table X.

---

5. How to run the ablation (Kaggle, GPU)

The runs must happen on a GPU with the datasets attached. On Kaggle:

5.1 New notebook → Settings: Accelerator **GPU**, Internet **On**, and Add Input the FER-2013 and RAF-DB datasets.

5.2 Clone the repo (own cell):

```
!git clone https://github.com/olfa-askri/FLEO.git
%cd /kaggle/working/FLEO
```

```
!git checkout main && git pull
!pip install -q ultralytics onnx onnxruntime
```

5.3 Find the dataset mounts, then prepare RAF-DB (paths vary — check first):

```
import os
for r,ds,fs in os.walk('/kaggle/input'):
    if r.count('/')-2 <= 2: print(r, sorted(ds)[:8])
```

Then, with the RAF-DB source path printed above (here it was `/kaggle/input/datasets/shuvoalok/raf-db-dataset`):

```
import subprocess
subprocess.run(['python','-m','data.prepare_rafdb',
                '--src','/kaggle/input/datasets/shuvoalok/raf-db-dataset',
                '--out','datasets/rafdb'])
print(os.listdir('datasets/rafdb'))   # -> ['images','labels','data.yaml']
```

5.4 Run the sweep — either open `notebooks/ablation_d.ipynb` and Run All, or one cell:

```
import subprocess, sys
for d in [4, 8, 16, 32]:
    print(f'=== d={d} ===', flush=True)
    subprocess.run([sys.executable,'-m','scripts.train',
        '--data','datasets/rafdb/data.yaml','--variant','fleo',
        '--cfg','yolov8n.yaml','--pretrained','yolov8n.pt','--d',str(d),
        '--epochs','40','--imgsz','128','--batch','16',
        '--device','0','--seeds','0','--workers','2',
        '--project',f'runs/abl_d{d}'])
```

5.5 Collect the table:

```
import pandas as pd, glob
rows=[]
for d in [4,8,16,32]:
    c=sorted(glob.glob(f'runs/abl_d{d}/**/results.csv',recursive=True))
    if not c: continue
    df=pd.read_csv(c[0]); df.columns=[x.strip() for x in df.columns]
    b=df.loc[df['metrics/mAP50(B)'].idxmax()]
    rows.append({'d':d,'ch':7*d,'mAP50':round(float(b['metrics/mAP50(B)']),4),
                 'mAP50-95':round(float(b['metrics/mAP50-95(B)']),4)})
print(pd.DataFrame(rows).to_string(index=False))
```

Copy the four mAP50 values into §3 and §4 above — the rebuttal is then complete.

Total time on a Kaggle P100/T4: ~2–3 h for the four runs (imgsz 128, 40 epochs).